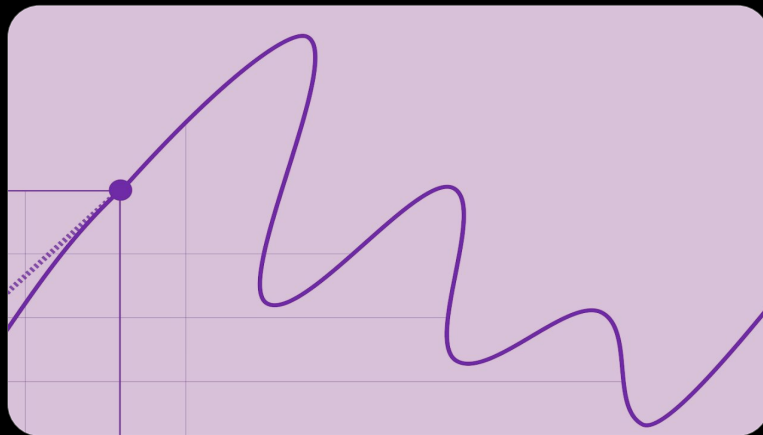


Padrão ViewModel

Prof. Dr. Fernando Sambinelli
IFSP Hortolândia

Vamos Começar

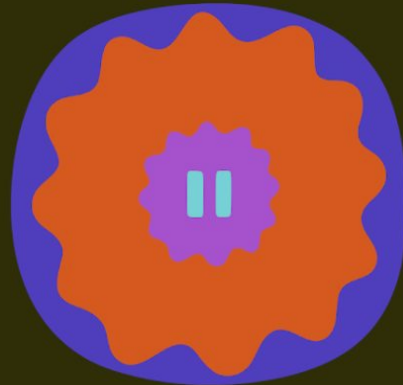


Padrão ViewModel

Componente que atua como um detentor de estado responsável por armazenar e expor os dados que a interface precisa exibir.

Como Funciona?

- O padrão segue uma arquitetura chamada UDF (Unidirectional Data Flow), onde os dados fluem em uma única direção.
- A interface gera um evento (como um clique de botão) e o transmite para o ViewModel.
- O ViewModel recebe esse evento, processa a lógica de negócio e atualiza o estado. Esse estado atualizado é então retornado à interface, que se recompõe para exibir as novas informações.
- Esse processo se repete continuamente conforme novos eventos ocorrem.



Estrutura Prática



- Na implementação moderna, você cria uma classe que estende `ViewModel` e usa `StateFlow` para gerenciar o estado.
- O padrão recomendado é usar uma classe de dados chamada `UiState` que encapsula todas as informações que a tela precisa exibir.

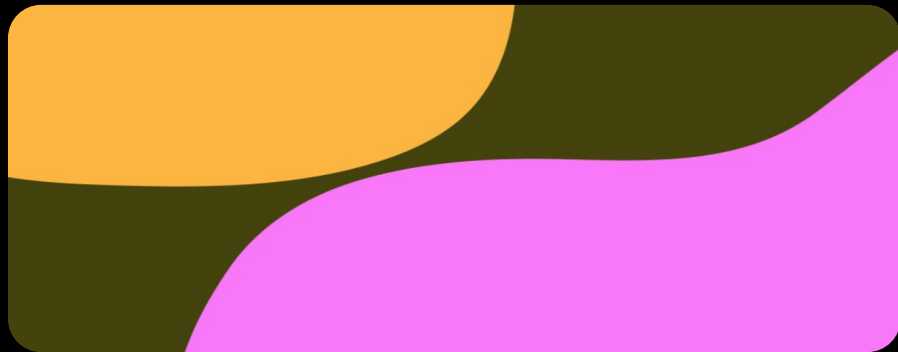
```
data class GameUiState(  
    val currentScrambledWord: String = "",  
    val score: Int = 0  
)  
  
class GameViewModel : ViewModel() {  
    private val _uiState = MutableStateFlow(GameUiState())  
    val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()  
  
    fun updateUserGuess(word: String) {  
        // lógica para atualizar o estado  
    }  
}
```

- No seu composável, você injeta o `ViewModel` usando a função `viewModel()` e converte o `StateFlow` em `State` do Compose com `collectAsState()` ou `collectAsStateWithLifecycle()`. Isso permite que a interface observe as mudanças e se recomponha automaticamente.

```
@Composable  
fun GameScreen(  
    gameViewModel: GameViewModel = viewModel()  
) {  
    val uiState by gameViewModel.uiState.collectAsStateWithLifecycle()  
    // Usar uiState para renderizar a tela  
}
```

Principais Vantagens

- ✧ O ViewModel persiste os dados mesmo quando a activity é destruída e recriada (como em rotações de tela), mantendo o estado intacto.
- ✧ Além disso, separa completamente a lógica de negócio da interface, tornando o código mais testável e reutilizável em diferentes sistemas de visualização.





Fernando Sambinelli

Professor e Doutor em Tecnologia
sambinelli@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo

